

Reviewer Pack — Secant Variety Dimension Lower Bound for Disjointness Commun...

Entry b6b0704f0dee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 23:26:09 UTC

Secant Variety Dimension Lower Bound for Disjointness Communication Complexity

Entry ID: b6b0704f0dee **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 23:26:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Secant Varieties) **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

For any $n \geq 1$, the communication complexity of the DISJOINTNESS function on n -bit inputs is at least the dimension of the secant variety of its communication matrix. This dimension is $\Omega(n)$ for all $n \geq 1$.

Rationale (proposer's reasoning):

Secant varieties capture the minimal number of rank-1 tensors needed to approximate a matrix, which mirrors the 'entanglement' required in communication protocols. The dimension of the secant variety provides a geometric lower bound on the complexity of the matrix's structure, directly linking algebraic geometry to communication complexity.

Taxonomy category: COMM_DISJ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cec0302781aa949e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_disjointness_matrix(n):
    if n <= 0:
        raise ValueError("n must be greater than 0")
    A = [[0] * (2**n) for _ in range(2**n)]
    for i in range(2**(n-1)):
        for j in range(i+1, 2**(n-1)):
            A[i][j] = 1
            A[j][i] = 1
    return A

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(M, b):
    n = len(M)
    M_b = [row + [b[i]] for i, row in enumerate(M)]

    for i in range(n):
        # Find the pivot
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j

        # Swap rows
        M_b[i], M_b[max_row] = M_b[max_row], M_b[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
```

```

        factor = M_b[j][i] / M_b[i][i]
        for k in range(n + 1):
            M_b[j][k] -= factor * M_b[i][k]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = M_b[i][-1] / M_b[i][i]
        for j in range(i-1, -1, -1):
            M_b[j][-1] -= M_b[j][i] * x[i]

    return x

def rank_of_matrix(M):
    n = len(M)
    M_copy = [row[:] for row in M]
    rank = 0
    for i in range(n):
        if M_copy[i][i] != 0:
            rank += 1
            for j in range(i+1, n):
                factor = M_copy[j][i] / M_copy[i][i]
                for k in range(n):
                    M_copy[j][k] -= factor * M_copy[i][k]
        else:
            found_nonzero = False
            for j in range(i+1, n):
                if M_copy[j][i] != 0:
                    M_copy[i], M_copy[j] = M_copy[j], M_copy[i]
                    rank += 1
                    found_nonzero = True
                    break
            if not found_nonzero:
                continue

        for j in range(n):
            if j != i and M_copy[j][i] != 0:
                factor = M_copy[j][i] / M_copy[i][i]
                for k in range(n):
                    M_copy[j][k] -= factor * M_copy[i][k]

    return rank

def secant_variety_dimension(matrix, n):
    identity = [[1 if i == j else 0 for j in range(2**n)] for i in range(2**n)]
    A = matrix
    B = identity
    count = 0

    while True:
        C = matrix_multiplication(A, B)
        rank_C = rank_of_matrix(C)

        if rank_C == n:
            return count

```

```

D = [[C[i][j] - A[i][j] for j in range(2**n)] for i in range(2**n)]
E = [[C[i][j] - B[i][j] for j in range(2**n)] for i in range(2**n)]

if rank_of_matrix(D) == n and rank_of_matrix(E) == n:
    count += 1
    A, B = C, identity
else:
    break

return count

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        try:
            A = generate_disjointness_matrix(n)
            secant_dim = secant_variety_dimension(A, n)

            if secant_dim < n:
                conjecture_holds = False
                counterexample = f"n={n}, secant dimension={secant_dim}"
                break

            total_metric_value += secant_dim
            instances_tested += 1
        except Exception as e:
            print(f"Exception in run_trial with seed {seed} and n={n}: {e}")

    return {
        "metric_name": "secant_variety_dimension",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results if result["instances_tested"] > 0)
    instances_tested = sum(result["instances_tested"] for result in results if result["instances_tested"] > 0)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={total_metric_value / instances_tested} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

riety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'secant_variety_dimension', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': None}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3990e9c1.py", line 170, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    ^^^^^^

```

NameError: name 'result' is not defined. Did you mean: 'results'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'result' before producing valid data. The counterexample mentioned in trials cannot be verified due to test failure. | next: Fix the test implementation to resolve the NameError and re-run with proper seeding

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	142340
2	propose	ollama_remote	qwen3:8b	0	0	52648
3	propose	ollama_remote	qwen3:8b	0	0	117974
4	propose	ollama_remote	qwen3:8b	0	0	138470
5	propose	ollama_remote	qwen3:8b	0	0	147012
6	propose	ollama_remote	qwen3:8b	0	0	93790
7	preregistration	ollama_remote	qwen3:8b	0	0	31724
8	novelty	ollama_remote	qwen3:8b	0	0	27491
9	novelty	ollama_remote	qwen3:8b	0	0	18467
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12223
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30050
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12051
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18537
14	judge	ollama_remote	qwen3:8b	0	0	115330

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 958107 ms total latency. Provider mix: {'ollama_remote': 14}

(full prompt+response transcripts available in `research/audit/b6b0704f0dee.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b6b0704f0dee.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b6b0704f0dee.tar.gz` (if generated)